

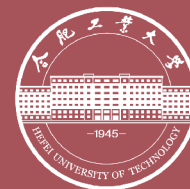
MiniDB：轻量级关系数据库系统

数据库系统课程设计

汇报人：张三

计算机科学与技术 23-1 班

指导老师：李四 教授



合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY

目录

CONTENTS



- 1 项目背景 ↗
- 2 系统设计 ↗
- 3 核心实现 ↗
- 4 测试与验证 ↗
- 5 总结与展望 ↗

1. 项目背景

1.1 研究背景与动机

数据库系统的重要性

- **数据管理核心**：关系数据库是现代信息系统的基石
- **教学意义**：实现一个数据库系统有助于深入理解内部原理
- **能力培养**：涵盖存储管理、索引、查询优化等关键知识

📖 通过"造轮子"的方式掌握数据库系统的核心机制

课程学习目标

- **SQL 支持**：实现基础 DDL 与 DML 语句解析执行
- **存储引擎**：基于页式存储的持久化数据管理
- **索引结构**：B+ 树索引加速查询检索
- **缓冲池**：内存页面缓存减少磁盘 I/O

📖 实现一个麻雀虽小五脏俱全的关系数据库

1.2 项目范围

支持的功能

- **DDL**: CREATE TABLE / DROP TABLE
- **DML**: INSERT / SELECT / UPDATE / DELETE
- **索引**: B+ 树索引的创建与查询
- **WHERE 子句**: 等值查询与范围查询
- **事务**: 单语句级别的原子性保证

技术选型

方面	选择
开发语言	C++ 17
构建工具	CMake
存储格式	定长页式存储
索引结构	B+ 树
测试框架	Google Test

2. 系统设计

2.1 总体架构

分层架构

- **SQL 解析层**：词法分析 + 语法分析
- **执行引擎层**：查询计划生成与执行
- **存储引擎层**：页面管理与索引操作
- **缓冲池层**：内存页面缓存与淘汰

“ 各层职责清晰，通过接口解耦



图1: MiniDB 总体架构图

2.2 存储引擎设计

页式存储模型

- **固定页大小**：每页 4KB，对齐磁盘扇区
- **页面类型**：数据页、索引页、元数据页
- **记录格式**：定长记录，槽式页面组织
- **空闲管理**：空闲页链表追踪可用空间

💧 数据文件以页为单位进行读写，减少随机 I/O

数据页内部结构

组成部分	大小	说明
页头	64B	页号、记录数、空闲偏移
槽目录	可变	每条记录的偏移量
空闲空间	可变	待插入区域
记录区域	可变	实际数据存储

💧 槽目录从页头向下增长，记录区域从页尾向上增长

2.3 查询处理流程

词法分析：将 SQL 字符串分割为 Token 序列（关键字、标识符、字面量）

语法分析：递归下降解析器构建抽象语法树（AST）

语义检查：验证表名、列名是否存在，类型是否匹配

计划生成：根据 AST 生成物理执行计划（扫描 / 索引查找）

计划执行：执行引擎调用存储层接口完成数据操作

结果返回：格式化输出查询结果或操作状态

2.4 缓冲池策略

LRU 页面淘汰策略

- **缓冲池容量**: 可配置的页面帧数 (默认 1024 帧)
- **页面映射**: 哈希表维护页号到帧号的映射
- **淘汰算法**: LRU 链表追踪页面访问顺序
- **脏页回写**: 淘汰脏页时先刷盘再释放帧

“ 缓冲池是减少磁盘 I/O 的关键组件，命中率直接影响整体性能



C

图2: 缓冲池工作原理示意图

3. 核心实现

3.1 B+ 树索引 - 数据结构

B+ 树节点设计

- **内部节点**：存储键值与子节点指针，不存数据
- **叶子节点**：存储键值与记录指针（RID）
- **叶链表**：叶子节点通过兄弟指针串联
- **阶数配置**：根据页大小自动计算最大阶数

“ 叶子节点串联支持高效的范围查询扫描



C

图3: B+ 树索引结构示意图

3.1 B+ 树索引 - 核心操作

插入操作

```
void BPlusTree::Insert(  
    const Key& key, const RID& rid) {  
    auto leaf = FindLeaf(key);  
    if (!leaf->IsFull()) {  
        leaf->InsertEntry(key, rid);  
    } else {  
        SplitAndInsert(leaf, key, rid);  
    }  
}
```

查找操作

```
bool BPlusTree::Search(  
    const Key& key, RID* result) {  
    auto leaf = FindLeaf(key);  
    return leaf->LookUp(key, result);  
}
```

“ **FindLeaf**: 从根节点沿内部节点二分查找，定位到目标叶子节点

3.2 缓冲池管理

页面获取逻辑

```
Page* BufferPool::FetchPage(
    page_id_t page_id) {
    if (page_table_.count(page_id)) {
        auto frame = page_table_[page_id];
        lru_.MoveToFront(frame);
        return &pages_[frame];
    }
    auto frame = lru_.Evict();
    FlushIfDirty(frame);
    disk_.ReadPage(page_id, &pages_[frame]);
    return &pages_[frame];
}
```

关键数据结构

组件	作用
page_table_	页号 → 帧号映射
lru_list_	LRU 淘汰链表
pages_[]	页面帧数组
disk_manager_	磁盘读写接口

“ 缓冲池对上层透明，执行引擎通过页号获取页面，无需关心是否命中缓存

3.3 SQL 解析器

词法分析器

- 使用手写的有限状态自动机
- 识别关键字、标识符、数字、字符串字面量
- 跳过空白字符与注释内容
- 输出 Token 流供语法分析器消费

📖 手写词法分析器相比正则引擎更轻量可控

语法分析器

- 递归下降解析，每条语法规则对应一个函数
- 支持 SELECT / INSERT / UPDATE / DELETE / CREATE
- 生成类型化的 AST 节点
- 语法错误时报告行号与期望 Token

📖 递归下降法结构清晰，便于扩展新语法

3.4 执行引擎

火山模型 (Volcano Model)

- 迭代器接口：每个算子实现 `Open/Next/Close`
- 按行拉取：上层算子调用下层 `Next()` 获取一行
- 算子类型：SeqScan、IndexScan、Filter、Projection

```
class Executor {
    virtual void Open() = 0;
    virtual bool Next(Tuple* t) = 0;
    virtual void Close() = 0;
};
```

执行流程示例

SELECT 语句的算子组合：

层级	算子	说明
顶层	Projection	选择输出列
中层	Filter	WHERE 条件过滤
底层	SeqScan	全表顺序扫描

“ 有索引时底层替换为 IndexScan，减少扫描行数

4. 测试与验证

4.1 功能测试

DDL 测试用例

- **建表**: 验证各数据类型 (INT / VARCHAR / FLOAT)
- **删表**: 验证表文件与元数据正确清除
- **重复建表**: 验证返回错误提示
- **列约束**: 验证 NOT NULL、PRIMARY KEY

 共 12 组 DDL 测试用例, 全部通过

DML 测试用例

- **INSERT**: 单行插入、批量插入、类型校验
- **SELECT**: 全表扫描、条件查询、索引查询
- **UPDATE**: 单列更新、多列更新、条件更新
- **DELETE**: 条件删除、全表删除

 共 28 组 DML 测试用例, 全部通过

4.2 性能测试

测试环境与结果

测试项	数据量	耗时
批量插入	10 万条	2.3s
全表扫描	10 万条	0.8s
索引等值查询	10 万条	0.02s
索引范围查询	10 万条	0.15s



图4：索引查询与全表扫描性能对比

索引查询相比全表扫描提速约 5-40 倍，验证了 B+ 树索引的有效性

5. 总结与展望

5.1 项目总结与未来方向

项目成果

- **存储引擎**: 实现了页式存储与定长记录管理
- **B+ 树索引**: 支持插入、查找、范围扫描
- **缓冲池**: LRU 淘汰策略有效降低磁盘 I/O
- **SQL 解析**: 支持基础 DDL 与 DML 语句

📖 完整实现了一个轻量级关系数据库的核心功能链路

未来改进方向

- **JOIN 支持**: 实现 Nested Loop Join 与 Hash Join
- **WAL 日志**: Write-Ahead Logging 保证事务持久性
- **并发控制**: 基于两阶段锁的多线程事务支持
- **查询优化**: 基于代价估计的执行计划选择

📖 后续可逐步完善, 向教学级完整数据库系统演进

感谢观看！



合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY